Riyadh Mobility Intelligence Starter Kit

Build and Deployment Workbook

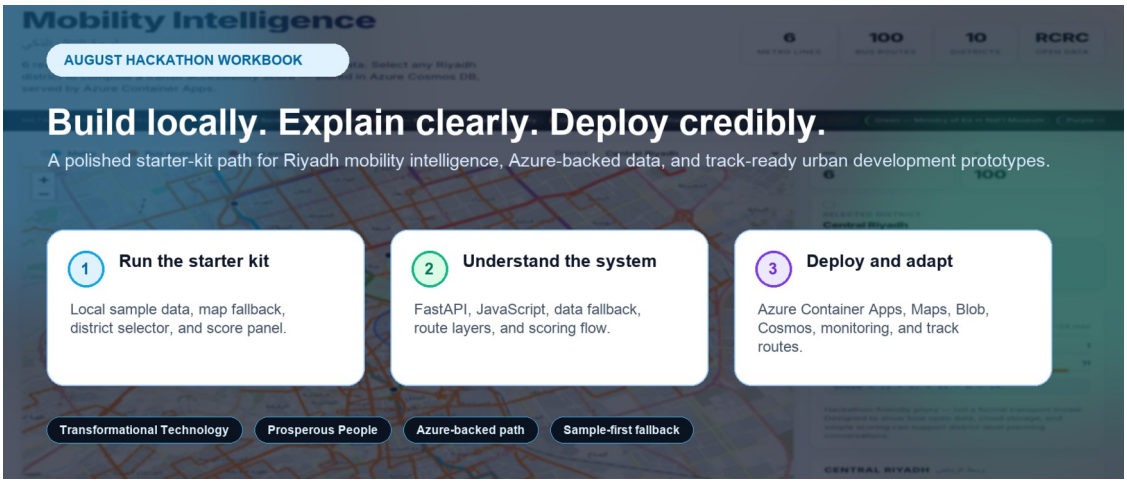
Prepared for atomcamp Arabia and the Riyadh Urban Development Hackathon in collaboration with Microsoft

2026-06-03

This working document helps August hackathon teams rebuild, run, adapt, and deploy the Riyadh Mobility Intelligence starter kit for Riyadh urban development use cases.

Repository: github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local app: http://127.0.0.1:8000
Deployed app: Paste `WEB_APP_URL` from `azd up` output here
Resource index: see the appendix for Azure, deployment, and project references.

Program context: atomcamp Arabia, Microsoft collaboration, Riyadh Urban Development Hackathon, August builder workbook.



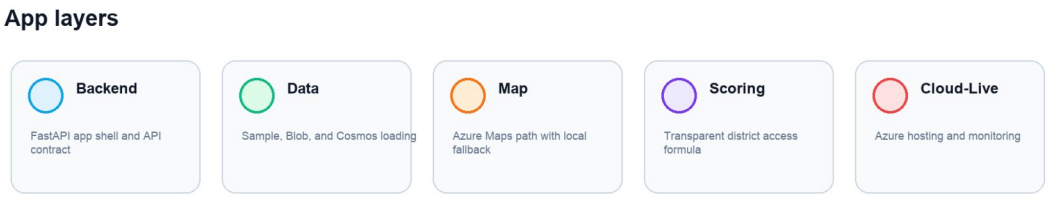
How to Use This Guide

This is a build workbook for hackathon builders, mentors, reviewers, and teams adapting the scaffold during the August hackathon. It is not a formal transport model or a production planning system. It is a practical starter kit that shows how open mobility data, a simple scoring API, map layers, and Azure services can fit together in a credible prototype.

Use the guide in this order:

1. Understand what the starter kit is building.
2. Run the app locally with bundled sample data.
3. Inspect the API and data-status endpoints.
4. Select a district and review the scoring panel.
5. Understand the backend, frontend, and data fallback chain.
6. Deploy the same app shape to Azure when ready.
7. Adapt the scaffold for a Riyadh Urban Intelligence Lab track.

The fastest successful path is local-first. Get the dashboard working from sample data before adding cloud credentials, Blob Storage, Cosmos DB, or a deployed app URL.



App layers

Figure: the starter kit is easiest to explain as five layers: backend, data, map, scoring, and cloud-live deployment.

1. What You Are Building

The application is a **Mobility Intelligence Starter Kit** for Riyadh. It lets a builder open a map, view public transport layers, select a district, and receive a transparent mobility access score. The same scaffold can later support district intelligence, sustainability overlays, culture routes, or other urban challenge ideas.

The starter kit includes:

- Riyadh metro lines and bus routes on a browser map.
- District selection and district-level mobility scoring.
- Mock live mobility events such as delay or congestion markers.
- Fallback-aware data loading that works without Azure credentials.
- Optional Azure-backed deployment using Azure Maps, Blob Storage, Cosmos DB, Container Apps, and Application Insights.
- A reusable pattern for teams that want to replace the data and scoring logic with another urban intelligence use case.

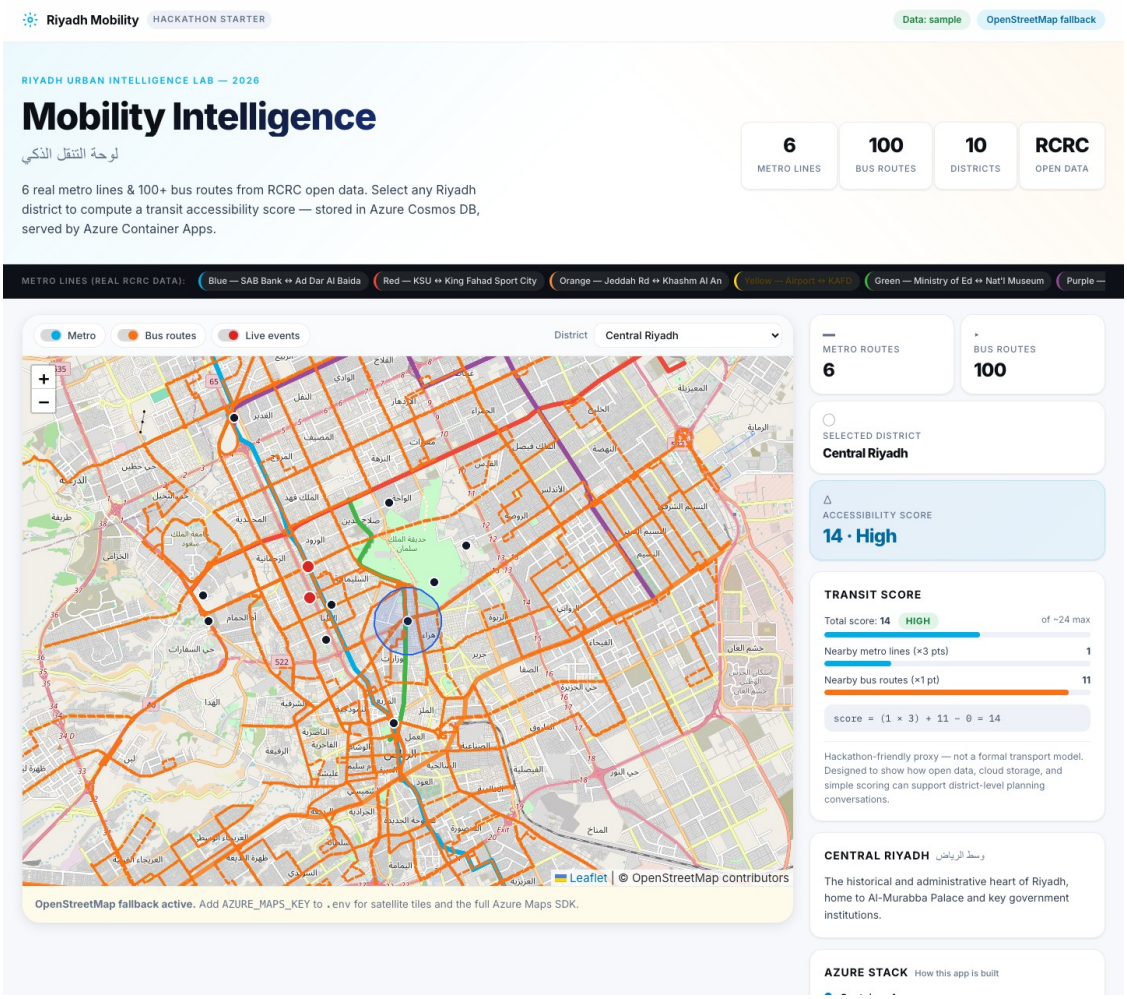
The product story is intentionally simple:

> If a user selects a Riyadh district, can the app show nearby mobility infrastructure, explain route coverage, and turn that into a clear starter score?

The current score is a teachable proxy:

```
score = (nearby metro count x 3) + nearby bus count - live delay penalty
```

This formula is not the final planning answer. It is a clear starting point for showing how data, scoring, API design, map layers, and Azure services connect.



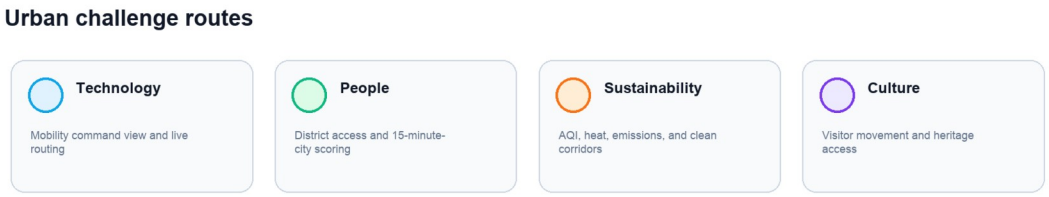
Local dashboard screenshot

Figure: local dashboard running from bundled sample data with metro lines, bus routes, district selector, score panel, and Azure service explainer visible.

2. Urban Challenge Track Mapping

The strongest primary fit is **Transformational Technology** because the current app already focuses on mobility visibility, route overlays, and cloud-backed intelligence. The strongest secondary fit is **Prosperous People** because the district scoring pattern can become a 15-minute city, walkability, or service-access score.

Sustainable Solutions and Culture should be treated as extension routes after the core app works.



Urban challenge routes

Track	How this starter kit fits	What to add next	Likely Azure services
Transformational Technology	Mobility command view, route overlays, delay markers, congestion proxy	parking demand, live route status, peak-load simulation	Container Apps, Azure Maps, Event Hubs, Stream Analytics, App Insights
Prosperous People	District access score, walkability proxy, 15-minute city thinking	service access layers, district comparison, equity indicators	Azure Maps, Cosmos DB, Blob Storage, Azure OpenAI for explanations
Sustainable Solutions	Mobility plus AQI, heat, emissions, and clean corridors	AQI stations, heat-stress routing, emissions overlay	Azure Maps, Blob Storage, Cosmos DB, Power BI
Culture	Visitor movement, event-day access, heritage route planning	heritage markers, crowd-sensitive routes, multilingual notes	Azure Maps, Cosmos DB, App Insights, optional translation services

Builder pitch:

```
Build a Riyadh mobility intelligence prototype with public route data,
district scoring, Azure Maps, and an Azure-backed live path. Use it directly
for Transformational Technology, or extend the same scaffold into Prosperous
People, Sustainable Solutions, or Culture use cases.
```

3. Services and Tools Map

This project is easiest to understand when each service or tool has one clear job.

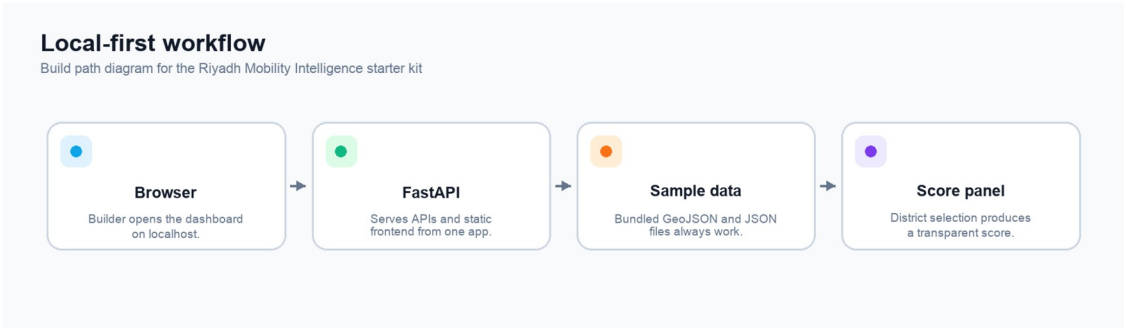
Service or tool	Role in this starter kit	Where builders see it
FastAPI	Serves backend API endpoints and the static frontend from one Python app	app/main.py, app/routes.py
Vanilla JavaScript	Coordinates page boot, map rendering, toggles, panels, and API calls	app/static/src/
Azure Maps	Provides the cloud map SDK and map services when configured	AZURE_MAPS_KEY, app/static/src/map.js
Azure Blob Storage	Stores processed GeoJSON/JSON files for cloud-backed data mode	scripts/upload_to_blob.py, app/data_access.py
Azure Cosmos DB	Stores app-shaped route, district, and live-event records	scripts/seed_cosmos.py, app/data_access.py
Azure Container Apps	Hosts the FastAPI app and frontend container	azure.yaml, infra/main.bicep
Azure Container Registry	Stores the built container image used by Container Apps	Bicep modules under infra/modules/
Application Insights	Tracks requests, failures, latency, and health for the deployed app	Azure portal after deploy
Log Analytics	Stores logs and diagnostics for Azure resources	Azure portal after deploy
Bicep	Defines infrastructure-as-code for the Azure environment	infra/main.bicep, infra/modules/
Azure Developer CLI	Creates and deploys the app environment with azd up	azure.yaml, bash scripts/deploy_azure.sh

4. Architecture and Build Path

The app is designed to work locally first, then graduate into an Azure-backed live path. The core principle is simple: **the app must remain useful even when Azure credentials, remote services, or live data are unavailable.**

Local-First Workflow

Local mode is the first build milestone. It should work on a builder laptop with no Azure account configured.

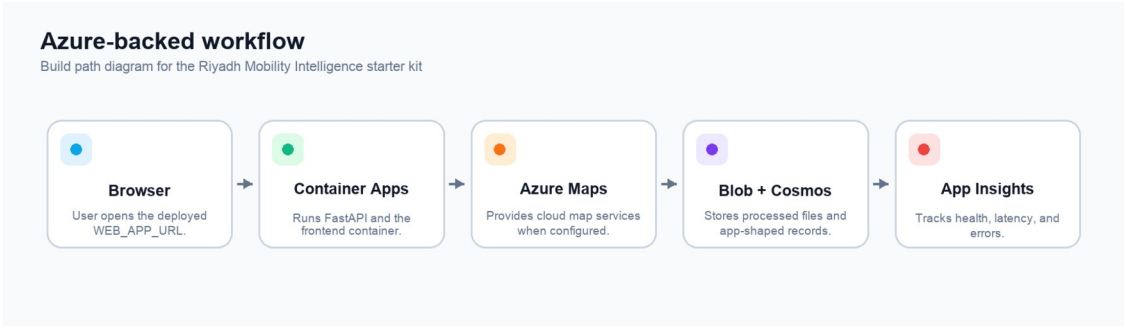


Local-first workflow

Local mode uses the same UI and API shape that the cloud deployment uses. The difference is the data source: bundled sample files replace cloud-backed records.

Azure-Backed Workflow

When deployed, the same containerized app runs in Azure Container Apps and can connect to Azure services for maps, storage, records, and monitoring.

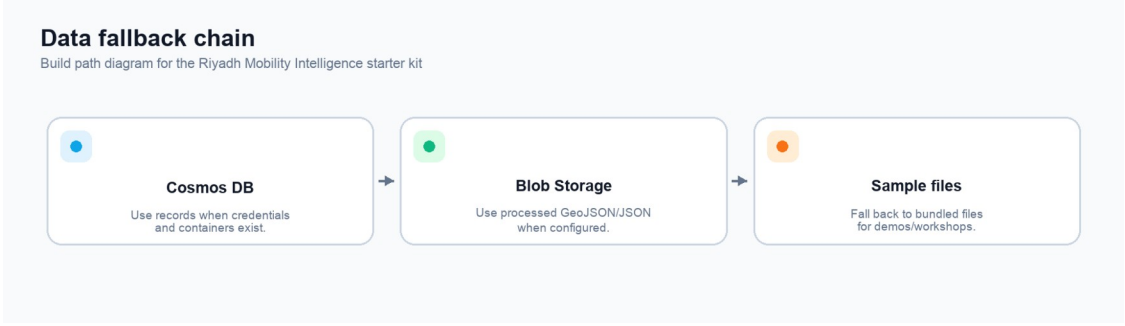


Azure-backed workflow

Use this path when the team is ready to show a credible cloud deployment or run post-deploy smoke tests.

Data Fallback Chain

The app should not fail just because cloud data is missing. Data access is intentionally fallback-aware.



Data fallback chain

In practice, builders should start with `DATA_MODE=sample`, then move to Blob or Cosmos only after the local app is working.

5. Project Structure

Start with the app shell, API, frontend, sample data, deployment files, and tests.

```
riyadh-mobility-intelligence-dashboard/
├── app/
│   ├── main.py                  # FastAPI app shell and static serving
│   ├── routes.py                # API endpoints
│   ├── data_access.py           # sample, Blob, Cosmos data loading
│   ├── scoring.py               # district mobility score
│   ├── azure_clients.py         # Azure SDK client setup
│   ├── config.py                # environment variables and defaults
│   └── static/
│       ├── index.html           # single-page dashboard shell
│       ├── src/                 # frontend JavaScript and CSS
│       └── sample-data/         # always-on sample files
├── scripts/                     # fetch, normalize, upload, seed, validate
├── infra/                       # Bicep infrastructure modules
├── docs/                        # build guide and supporting notes
├── tests/                       # scoring, API, and data-shape checks
├── azure.yaml                   # Azure Developer CLI project
├── Dockerfile                   # container definition
└── requirements.txt             # Python dependencies
```

Area	Start here	Why it matters
Backend entrypoints	app/main.py, app/routes.py	FastAPI app shell, static serving, API contract
Data loading	app/data_access.py, app/static/sample-data/	sample, Blob, and Cosmos fallback behavior
Score formula	app/scoring.py	transparent scoring logic that teams can adapt
Frontend	app/static/index.html, app/static/src/main.js	page shell, boot sequence, API orchestration
Map behavior	app/static/src/map.js, app/static/src/layers.js	Azure Maps path, local fallback, overlays
Deployment	azure.yaml, infra/main.bicep, scripts/deploy_azure.sh	cloud-live path
Tests	tests/	API, data-shape, health, and scoring checks

Where the Data Comes From

The bundled sample data is derived from Riyadh Commission for Riyadh City open geospatial datasets. The files live under `app/static/sample-data/` and keep the app useful even when no cloud services are configured.

File	Contents	Records
riyadh_metro_lines_sample.geojson	Metro line features with names, colors, and geometry	6
riyadh_bus_routes_sample.geojson	Bus route features with route IDs and geometry	100
district_centers_sample.geojson	Riyadh district center points with English and Arabic names	10
mock_live_events_sample.json	Sample delay or incident markers	variable

Fresh data workflow:

```
python3 scripts/fetch_rcrc_data.py
python3 scripts/normalize_to_geojson.py
python3 scripts/validate_data.py
python3 scripts/upload_to_blob.py              # optional cloud step
PYTHONPATH=. python3 scripts/seed_cosmos.py    # optional cloud step
```

6. Run Locally

Local setup is deliberately direct. The goal is to get one useful screen running before adding cloud services.

Play-by-Play

```
git clone https://github.com/Esturban/microsoft-hackathon-riyadh-
mobility.git
cd microsoft-hackathon-riyadh-mobility
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env
```

Open `.env` and confirm the default local mode:

```
DATA_MODE=sample
```

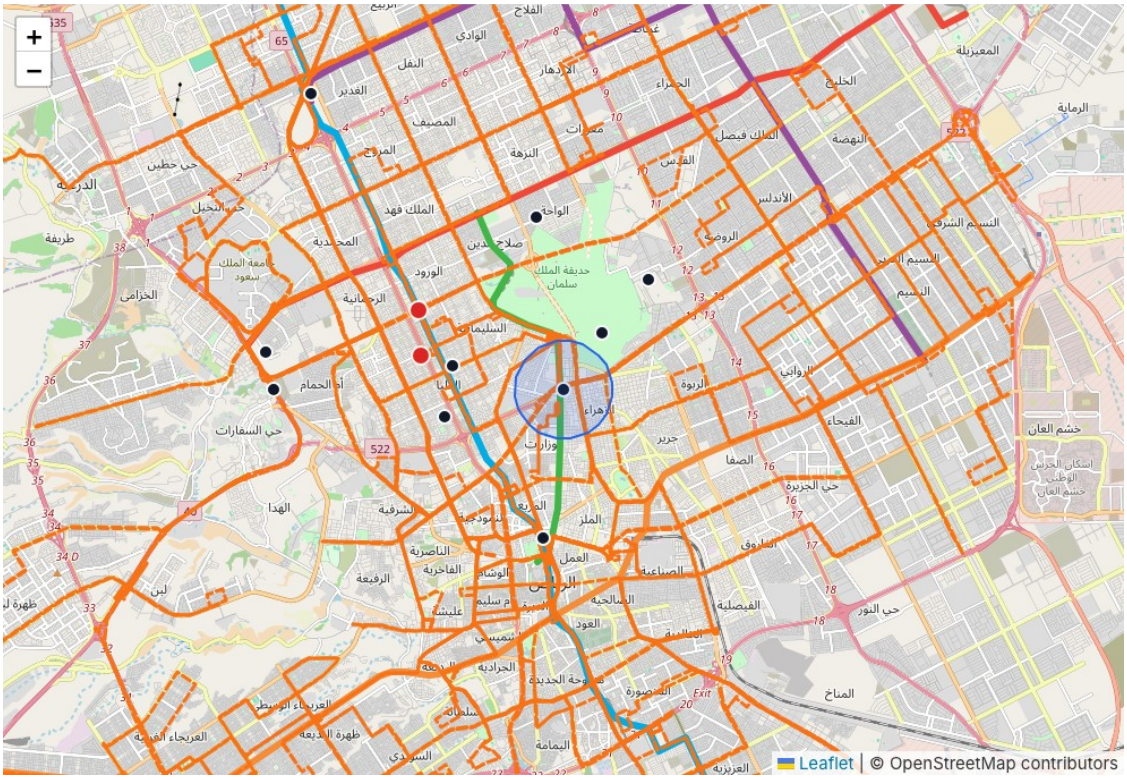
Start the app:

```
python -m uvicorn app.main:app --reload
```

Open the dashboard:

```
http://127.0.0.1:8000
```

The first page load can take time. Wait for the district selector, layer controls, map container, and score panel area before deciding whether the app is loaded. If Azure Maps is not configured, the app should use the OpenStreetMap fallback and still show the route overlays.



Map layers screenshot

Figure: metro and bus layers rendered locally with OpenStreetMap fallback tiles.

Local Verification Checklist

- Dashboard loads at `http://127.0.0.1:8000`.
- Map renders using Azure Maps or the OpenStreetMap fallback.
- Metro and bus layers appear.
- District selector has 10 Riyadh districts.
- Selecting a district updates the score panel.
- `/health` returns `{"status":"ok"}`.
- `/api/data-status` reports active sample mode.

Useful checks:

```
curl -s http://127.0.0.1:8000/health
curl -s http://127.0.0.1:8000/api/data-status | python -m json.tool
python -m pytest
python scripts/validate_data.py
```

7. Backend Walkthrough

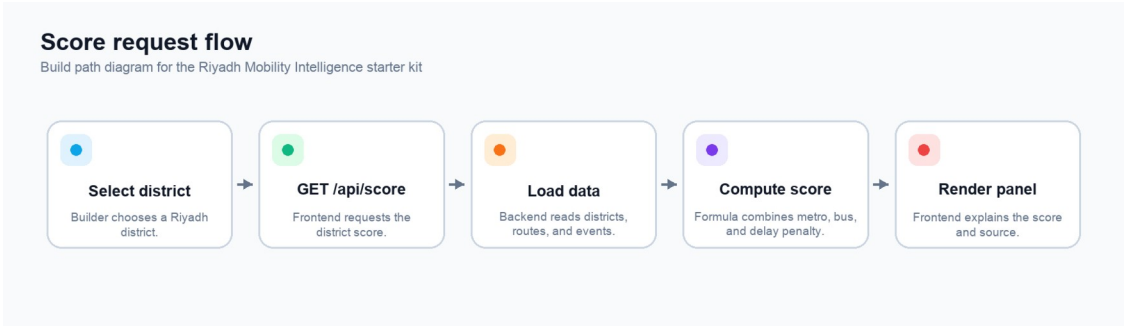
The backend is a small FastAPI application. It serves both the API and the static frontend so the starter kit stays easy to run and easy to deploy.

Endpoint	Purpose	What a builder should look for
GET /health	Confirms the app is alive	{"status":"ok"}
GET /api/config	Sends runtime settings to the frontend	app name, map mode, access buffer, data mode
GET /api/routes	Returns route summaries and KPI counts	metro and bus route counts
GET /api/routes/geojson?mode=metro	Returns map-ready metro geometry	GeoJSON payload and source
GET /api/routes/geojson?mode=bus	Returns map-ready bus geometry	GeoJSON payload and source
GET /api/districts	Returns district selector records	10 district items in sample mode
GET /api/score?districtId=...	Returns score and score components	score, rating, metro count, bus count, penalty
GET /api/live-events	Returns mock or configured event overlays	delay/event items and source
GET /api/data-status	Explains active data source and fallback status	current mode, counts, fallback message

API data status screenshot

Figure: `/api/data-status` confirms sample mode and explains the fallback path.

Score Request Flow



Score request flow

The score formula stays on the backend so it can be tested and reused:

```
score = (nearby metro count x 3) + nearby bus count - live delay penalty
```

Keep this formula easy to explain. Teams can later replace the weights or add new inputs such as parking, walkability, public services, heat, or event density.

8. Frontend Walkthrough

The frontend is a single-page application built with Vanilla JavaScript. This keeps the starter kit accessible to mixed-experience teams while still showing a complete app workflow.

On startup, `app/static/src/main.js` fetches the core runtime data in parallel:

```
const [config, routes, metro, bus, districts, events, dataStatus] = await
Promise.all([
  api.getConfig(),
  api.getRoutes(),
  api.getRouteGeojson("metro"),
  api.getRouteGeojson("bus"),
  api.getDdistricts(),
  api.getLiveEvents(),
  api.getDataStatus(),
]);
```

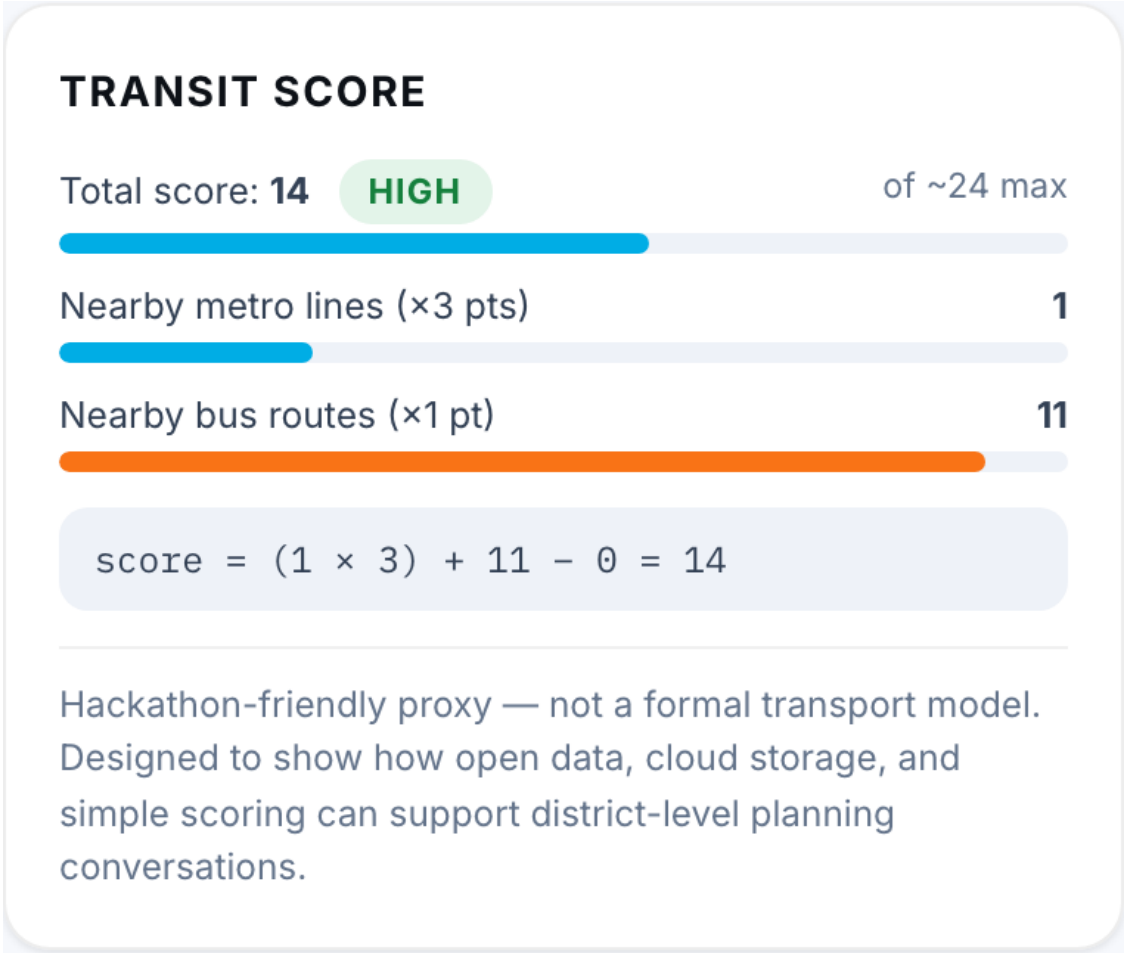
The map layer system makes the city data visible and explainable.

Layer	What it shows	Why it matters
Metro lines	high-capacity mobility corridors	shows structural transit coverage
Bus routes	surface transit coverage	shows fine-grained network reach
Districts	selectable district points	anchors the score conversation
Live events	delay or incident markers	demonstrates cloud-live extension potential
Accessibility buffer	rough selected-district service area	makes the score formula visible

District selector screenshot

The score panel translates raw counts into a judge-friendly story:

- selected district name
- score number
- rating
- nearby metro count
- nearby bus count
- live-event penalty
- readable formula
- active data source context



District score panel screenshot

Figure: district score panel after selecting a Riyadh district.

9. Cloud-Live Deployment

The cloud-live path is the deployable version of the same starter kit. It is useful when a team needs to show that the app can move beyond a laptop and into a credible Azure environment.

Prerequisites

Before deploying, confirm:

- Azure account is available.
- Azure CLI is installed.
- Azure Developer CLI is installed.
- Shell is authenticated to the right Azure tenant and subscription.
- Target location and resource group are known.
- Local app and tests already pass.

Sign in:

```
az login
azd auth login
azd init
```

Deploy with the helper:

```
bash scripts/deploy_azure.sh
```

Optional location and resource group:

```
bash scripts/deploy_azure.sh eastus rg-riyadh-ud-eastus
```

The helper prints active environment values, then runs the Azure Developer CLI deployment workflow.

What Deployment Creates

Azure resource	Role
Azure Container Apps	hosts FastAPI and the static frontend
Azure Container Registry	stores the built container image
Azure Maps account	supports cloud map services
Azure Blob Storage	stores raw and processed mobility files
Azure Cosmos DB	stores route, district, and event records
Application Insights	tracks health, latency, failures, and requests
Log Analytics	stores logs and diagnostics

After deployment:

1. Copy `WEB_APP_URL` from `azd` output.
2. Open the deployed app in a browser.

3. Check `<WEB_APP_URL>/health`.
4. Check `<WEB_APP_URL>/api/data-status`.
5. Confirm the active data mode is expected.
6. Upload processed files to Blob if using Blob mode.
7. Seed Cosmos DB if using Cosmos mode.
8. Inspect App Insights if the deployed app fails or loads slowly.

Optional cloud data steps:

```
python3 scripts/upload_to_blob.py
PYTHONPATH=. python3 scripts/seed_cosmos.py
```

Teardown when finished:

```
bash scripts/destroy_resource_group.sh --yes
```

The teardown script deletes the current Azure resource group and intentionally requires `--yes`.

10. Starter Kit Adaptation Routes

Treat this codebase as a scaffold. Teams should keep the working shell, replace the domain data, and adapt the score or map layers toward the track they are pursuing.

Route 1: Mobility Command View

Best fit: Transformational Technology.

Keep:

- route layers
- event overlay
- district selector
- route KPIs
- data-status debugging

Replace or add:

- real congestion or delay source
- parking demand layer
- peak-load simulation
- route delay explanations

Likely Azure services: Azure Maps, Container Apps, Event Hubs, Stream Analytics, Cosmos DB, Application Insights.

Route 2: District Intelligence

Best fit: Prosperous People.

Keep:

- district selector
- scoring panel
- map focus behavior
- Cosmos-ready district records

Replace or add:

- service access layers such as clinics, schools, parks, or transit stops
- 15-minute city score
- walkability indicators
- district comparison panel

Likely Azure services: Azure Maps, Blob Storage, Cosmos DB, Container Apps, optional Azure OpenAI for plain-language score explanations.

Route 3: Sustainable Mobility Overlay

Best fit: Sustainable Solutions.

Keep:

- mobility layers
- selected district context
- route scoring pattern
- fallback-aware sample files

Replace or add:

- AQI stations
- heat layer
- pollution hotspot markers
- clean corridor recommendations
- emissions or low-carbon route scoring

Likely Azure services: Azure Maps, Blob Storage, Cosmos DB, Power BI, Application Insights.

Route 4: Culture and Visitor Movement

Best fit: Culture.

Keep:

- map layer structure
- event markers
- route overlays
- selected place or district panel

Replace or add:

- heritage site markers
- crowd-sensitive route suggestions
- multilingual visitor notes
- event-day access planning

Likely Azure services: Azure Maps, Cosmos DB, Blob Storage, Container Apps, optional translation services.

11. Demo Flow

Use this short flow when presenting to mentors or judges.

Step	Show	Explain
1	Dashboard landing view	This is a Riyadh mobility intelligence starter kit, not just a static map
2	Metro and bus toggles	The map layers show public transport coverage
3	District selector	District context drives the scoring workflow
4	Score panel	The score is transparent and easy to adapt
5	/api/data-status	The app is fallback-aware and cloud-live ready
6	Azure workflow diagram	Services map to hosting, maps, files, records, and monitoring
7	Adaptation routes	The same scaffold supports multiple hackathon tracks

Keep the live demo local unless the deployed app has already passed `/health` and `/api/data-status` checks.

Appendix

Common Commands

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
python -m uvicorn app.main:app --reload
python -m pytest
python scripts/validate_data.py
bash scripts/deploy_azure.sh
bash scripts/destroy_resource_group.sh --yes
```

Debugging Checklist

- If the page loads slowly, wait for the district selector, layer controls, map area, and score panel before judging success.
- If the map is blank, clear `AZURE_MAPS_KEY` in `.env` and reload so the local fallback can be used.
- If cloud data is missing, open `/api/data-status` first.
- If sample files fail, run `python scripts/validate_data.py`.
- If the deployed app fails, check `/health`, `/api/data-status`, Container App logs, and Application Insights.

Visual Asset Checklist

Asset	Included in this guide
app layer icon row	docs/assets/rebuild-guide/icon-row-app-layers.png
four-track icon row	docs/assets/rebuild-guide/icon-row-tracks.png
local dashboard screenshot	docs/assets/rebuild-guide/local-dashboard.png
map layer screenshot	docs/assets/rebuild-guide/map-layers.png
district selector screenshot	docs/assets/rebuild-guide/district-selector.png
district score panel screenshot	docs/assets/rebuild-guide/district-score-panel.png
data-status endpoint screenshot	docs/assets/rebuild-guide/api-data-status.png
local workflow diagram	docs/assets/rebuild-guide/diagram-local-first.png
Azure workflow diagram	docs/assets/rebuild-guide/diagram-azure-backed.png
fallback chain diagram	docs/assets/rebuild-guide/diagram-data-fallback.png
score request diagram	docs/assets/rebuild-guide/diagram-score-request.png

Resource Index

Use this page as the resource hub instead of repeating links in the footer. Keep the cover page focused on the repo, local app, and deployed app URL; keep this appendix focused on build, deployment, and Azure references.

Need	Open
GitHub repo	github.com/Esturban/microsoft-hackathon-riyadh-mobility
Local dashboard	http://127.0.0.1:8000
atomcamp Arabia	atomcamparabia.com
Azure Developer CLI docs	Microsoft Learn
azd up workflow	Microsoft Learn
Azure Container Apps docs	Microsoft Learn
Azure Maps docs	Microsoft Learn
Azure Cosmos DB docs	Microsoft Learn
Student quickstart	README.md
Architecture notes	docs/architecture.md
Azure deployment notes	docs/azure_deployment.md
Data source notes	docs/data_sources.md
Troubleshooting	docs/troubleshooting.md
Demo script	docs/judging_demo_script.md

Glossary

Azure-backed live path
The deployable path where the starter kit uses Azure services for maps, hosting, files, records, and monitoring.

Blob Storage
Azure file storage for raw and processed mobility datasets.

Cosmos DB
Azure document database for route summaries, district records, and event records.

FastAPI
Python web framework used to serve the API and static frontend.

GeoJSON
JSON format for map features such as points, lines, and polygons.

Mobility Intelligence Starter Kit
The reusable scaffold that teams can adapt into mobility, district, sustainability, or culture prototypes.

Sample fallback
The local files that keep the app useful even when cloud services or remote data are unavailable.

Single-page application
The current frontend shape: one main web page that loads data and updates panels dynamically.

Closing Note

The starter kit is strongest when it stays practical: one useful local screen, clear route layers, transparent scoring, and a cloud-live path that is credible without becoming heavy. Build the smallest version that can be explained well, then extend it toward the hackathon track that matters most.